

Here is an idea for a unique, potentially highly successful hyper-casual game, called "**Gravity Weaver.**"

🌟 Game Idea: Gravity Weaver

Pillar	Implementation	Success Factor Alignment
Core Mechanic	One-Touch Gravity Switch (Unique Twist on Rise/Fall)	Accessibility: Simple, one-finger control (Flappy Bird style).
Gameplay Loop	Endless, Procedural Obstacle Course with an increasing sense of speed.	Short Sessions: Quick, high-score driven runs.
Progression/Meta	Cosmetic Collection (Weaver Skins & Trails)	Engagement Loop: Drives retention and in-app purchases.
Virality	Leaderboards & Instant Death Replays	Social Driver: Encourages sharing frustrating/impossible fails.

📍 The Core Mechanic: Gravity Weaver

Concept

The player controls a small, continuous, winding "**Thread**" (the Weaver) moving horizontally across the screen. The goal is to guide the thread between a series of randomly generated obstacles (spikes, narrow gaps).

Control: One-Touch Gravity Flip

- Default State:** The Thread is subject to **downward gravity** and naturally sinks toward the bottom of the screen.
- Player Action:** A single **tap anywhere on the screen instantly reverses gravity**, causing the Thread to float upward.
- Release Action:** Releasing the screen instantly flips gravity back to downward.

This creates a rhythmic, alternating tap-and-release pattern to keep the Thread weaving through the narrow center path, blending the timing-based challenge of *Flappy Bird* with the continuous movement of *Subway Surfers*.

Monetization and Success Factors

1. High-Value In-App Purchases (IAPs)

- **Cosmetic Customization:** The primary purchase driver. Players can unlock unique Thread "**Skins**" (e.g., a lightning bolt, a snake, a candy cane) and "**Trail Effects**" (e.g., spark trails, smoke, or rainbow patterns). This is pure customization and gives no competitive advantage.
- **"Remove Ads" Subscription:** Offers the premium experience and a reliable recurring revenue stream.
- **Starter Bundles:** Limited-time, high-value packs combining premium currency and a rare skin.

2. Rewarded Video Ads

- **Revive Option:** Offer the player one **Revive** (to restart from the point of death) per run in exchange for watching a 30-second rewarded video ad. This leverages the "frustration loop" by giving the player a way to overcome their failure instantly.
- **Double Score:** Offer to double the score of a successful run by watching an ad after the run ends, appealing to high-score hunters.

3. Virality and Social Loop

- **Death Replay GIF/Video:** When the player inevitably crashes, the game instantly generates a short, satisfying clip of their failure and the spectacular collision. An "**Instant Share**" button encourages users to post this short GIF/video to social media with their high score. This drives viral acquisition, similar to *Flappy Bird*'s difficulty.
- **Weekly Challenges:** Introduce a simple **Global Leaderboard** and **Friend Leaderboards**, refreshing weekly to encourage competition and repeat play.

By focusing on a novel yet intuitive one-touch mechanic, high-satisfaction visuals, and a monetization strategy built on cosmetic wants and frustration relief, **Gravity Weaver** hits all the marks for modern mobile success.

The key to developing a successful hyper-casual game like **Gravity Weaver** is rapid prototyping and a heavy focus on the feel of the core mechanic.

Here is a step-by-step developer guide, using **Unity** (the industry-standard engine for mobile monetization) as the primary tool.

Phase 1: Rapid Prototyping (4-7 Days)

The only goal here is to prove the core mechanic is fun and satisfying.

Step 1: Tool Selection and Setup

- **Game Engine:** Use **Unity** (2D Project Template) or **Godot** (especially strong for 2D). Unity has superior documentation and support for ad monetization (a critical factor).
- **Core Assets:** Use simple colored shapes (circles, rectangles) for all elements—no fancy art yet!

Step 2: Implement Core Mechanics

- **The Weaver (Player):** Create a simple sprite (a continuous line or small circle) that moves automatically on the X-axis (forward). Give it a **Rigidbody2D** component.
- **Gravity Flip Script (C#):** In the **Update()** loop, check for screen tap/mouse click.
 - **On Tap/Hold:** Set the Rigidbody2D's gravity scale to a **positive value** (to make it float up).
 - **On Release:** Set the Rigidbody2D's gravity scale to a **negative value** (to make it sink down).
 - *Tweak the gravity scale values until the movement feels fluid and requires precise timing.*

Step 3: Create the Endless Loop

- **Obstacle Generation:** Create a script that randomly generates vertical pairs of obstacles (the spikes/gaps) off-screen and moves them toward the player at an increasing speed.
- **Collision:** If the Weaver collides with an obstacle, trigger a simple, instant **Game Over** screen.
- **Feedback:** Add satisfying **sound effects** when the gravity flips and a particle burst when the player successfully passes a gap.



Phase 2: Polish and Monetization (10-14 Days)

Once the prototype feels good, integrate the systems that drive retention and revenue.

Step 4: Visual and Audio Polish

- **Minimalist Art:** Implement the final minimalist art style (e.g., solid colors, gradients, smooth geometry).
- **Trails and Skins:** Create a simple **PlayerPrefs** system to save unlocked skins/trails. Add a **Shop UI** where players can select their active cosmetic.
- **Juice:** Improve the **camera shake** on near-misses, and make the collision explosion highly satisfying.

Step 5: Implement Monetization (The Revenue Engine)

- **Ad Network Integration:** Integrate a mobile ad SDK (e.g., **Unity Ads**, **Google AdMob**, or **IronSource**).
- **Rewarded Video (Most Important):**
 - Place a "RESTART WITH REVIVE? (Watch Ad)" button on the Game Over screen.
 - **C# Implementation:** Call the rewarded ad API. Upon successful completion of the ad, reward the user by resetting the game state to the point of death (e.g., move the Weaver back slightly) and continuing the run.
- **Remove Ads IAP:** Implement a standard In-App Purchase (IAP) service (like Unity IAP) for a one-time purchase to disable interstitial and banner ads.



Phase 3: Analytics and Export

This phase is non-negotiable for a hyper-casual game, as success is entirely determined by metrics.

Step 6: Analytics Tracking

- **Integrate Firebase Analytics/Unity Analytics:** This is crucial for tracking player behavior.
- **Track Key Metrics:** Use code to fire analytics events for the following actions:
 - `Game_Start`
 - `Game_Over` (include parameters for Score and Run Duration)
 - `Ad_Watched_Revive`
 - `Skin_Unlocked`
 - `IAP_Purchase`

Step 7: Exporting Data to Google Sheets

The simplest and most immediate way to get live data into Google Sheets for quick analysis is often using a **backend proxy (like Google Apps Script)** or a dedicated analytics export tool.

Success Factor Metric	Value
Monetization: Remove Ads IAP	<code>IAP_Purchase</code> event count
Monetization: Rewarded Video	<code>Ad_Watched_Revive</code> event count

Retention: Day 1 (D1) %	\$\%\$ of users who return 24 hours after installing.
Core Loop: Average Session Time	The average time (seconds) between Game_Start and Game_Over .
Virality/Difficulty	Average Score (High Score Distribution)

The Technical Export Path:

1. **Use a Middleware/API (Recommended):** Use a service like **Unity Analytics Raw Data Export** or set up **Firebase Analytics**. Both platforms provide dashboards and API access to download raw data (JSON or CSV).
2. **Google Sheets Integration:**
 - Set up a **Google Apps Script** as a web app.
 - In your Unity/C# code, use **UnityWebRequest** to send real-time **HTTP POST** requests containing your Game Over scores, ad views, etc., to your Google Apps Script URL.
 - The Apps Script function receives the JSON data and uses the Sheets API to **append a new row** to your designated Google Sheet.